MOSCOW AVIATION INSTITUTE
(NATIONAL RESEARCH UNIVERSITY)

Institute No. 8 “Computer Science and Applied Mathematics”
Department 806 “Computational Mathematics and Programming”

Laboratory work No. 1
Course “Computer Graphics”

3D Graphics Basics

Student: M. A. Khorokhorin
Group: M8O-306BV-24
Teacher: V. D. Bakharev

Moscow, 2026

Task

Goal: learn the basics of 3D graphics — building a simple 3D object and projecting it onto a 2D plane with a perspective projection matrix. The program runs in real time on Vulkan, GLFW and ImGui.

Variant 4: a truncated regular tetrahedron, whose faces are 4 regular triangles and 4 regular hexagons. The base task without extra tasks is done: the object rotates, and the rotation speed is changed in the interface.

Method

Geometry. The regular tetrahedron has the corners (1, 1, 1), (1, −1, −1), (−1, 1, −1), (−1, −1, 1). Every edge A_iA_j is split into three parts, the point near A_i is $P_{ij} = (2A_i + A_j) / 3$. This gives 12 vertices, 18 edges and 8 faces. Every corner of the tetrahedron is cut off as a triangle and every original face becomes a hexagon. The points are normalized, so the object fits into the unit sphere. Every face has its own vertices and color, a hexagon is split into 4 triangles as a fan, 20 triangles in total. The vertex order of a face is chosen so that the normal $(b − a) \times (c − a)$ points outwards.

Transformations. Every frame computes $MVP = P \cdot V \cdot M$. The model matrix M is a rotation by an angle that grows with time at the speed from the interface. The view matrix V is built with glm::lookAt, the perspective matrix P with glm::perspectiveRH_ZO and a 60° field of view, then the element P[1][1] changes its sign, because in Vulkan the screen Y axis points down and depth is in [0, 1].

Vulkan objects. The vertex and index buffers (VkBuffer) are allocated with Vulkan Memory Allocator and filled with memcpy. The MVP matrix is stored in a uniform buffer described by a VkDescriptorSetLayout, allocated from a VkDescriptorPool as a VkDescriptorSet and passed to the pipeline through a VkPipelineLayout. The graphics pipeline VkPipeline: vertices with position and color, a triangle list, back-face culling, depth test and write, dynamic viewport and scissor. GLSL shaders are compiled to SPIR-V with glslc.

Frame. update draws the ImGui window and computes the MVP matrix. render, after the fence of the previous frame is waited on, copies the matrix to the uniform buffer and records the command buffer: begin the render pass clearing color and depth, set the viewport and scissor, bind the pipeline, the buffers and the descriptor set, vkCmdDrawIndexed.

Results

The figures show the rotating object at different moments, the interface window with the rotation speed is in the top left corner.

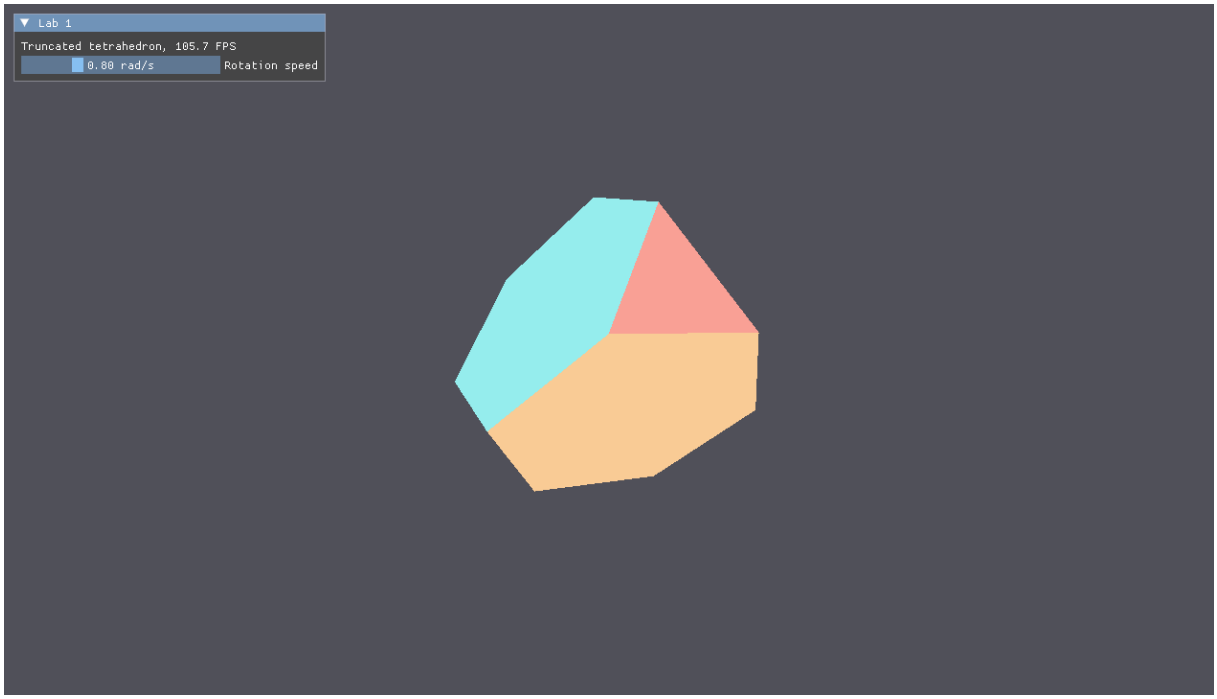


Figure 1 — truncated tetrahedron with two hexagonal faces and a triangular face

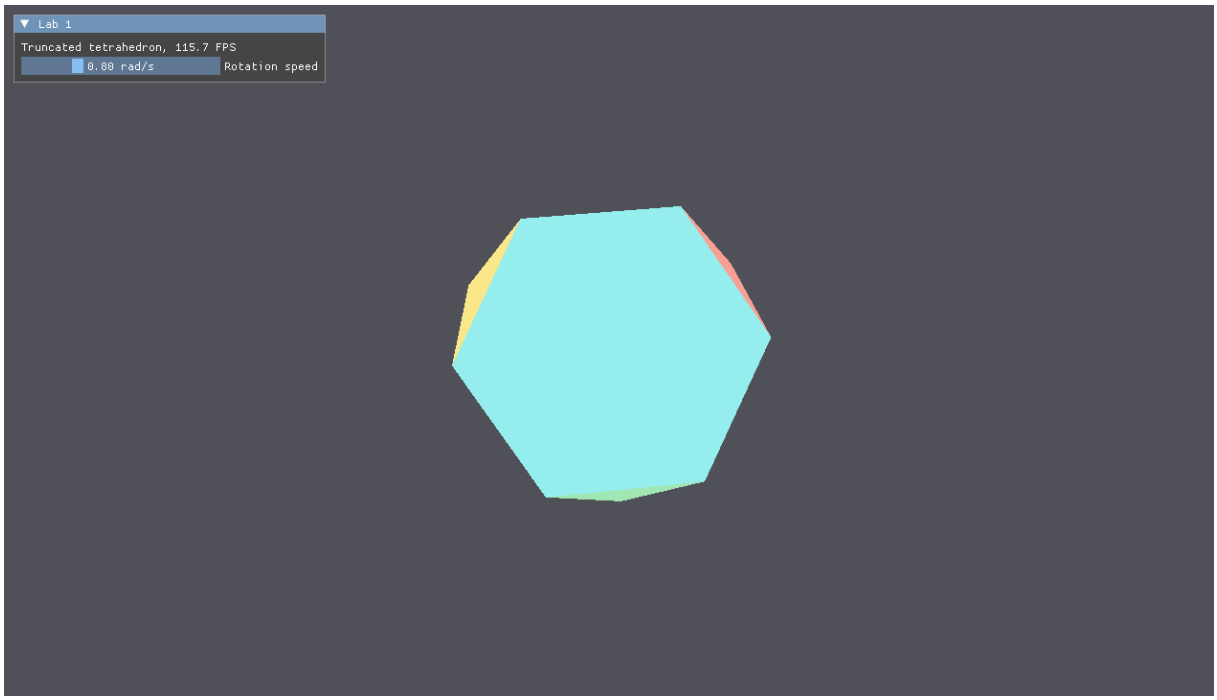


Figure 2 — a hexagonal face turned to the camera

Conclusions

I learned to build a 3D object from vertices and indices, pass a matrix to a shader through a uniform buffer and a descriptor set, describe the Vulkan graphics pipeline and project the scene with a perspective matrix for the Vulkan coordinate system.